

Sprint Retrospective Report

Restaurant Table Booking & Pre-Ordering App (RTBPOA)

PES University · Software Engineering Lab · Problem Statement #36 (Retail, E-Commerce & Finance)

Prepared by: Adyanth Mallur (SRN PES1UG24CS036), Section 5A

Jira project: RTBPOA — adyanthmallur.atlassian.net · Report date: September 4, 2026

1. Epics

The RTBPOA backlog is organised into four epics, each tracing to a functional/non-functional requirement and a set of use cases from the requirements-engineering phase.

Epic	Scope	Use cases / traces	Stories · points · status
Epic 1 Table Reservation & Floor Plan Management	Diners browse the live 2D floor plan, select/hold a table, and manage the reservation lifecycle including special requests and modifications or cancellations.	UC-01, UC-02, UC-06, UC-08 Traces: FR-001	1.1 (5) · 1.2 (3) · 1.3 (3) · 1.4 (2) 4 stories, 13 pts — all Done
Epic 2 Pre-Order & Kitchen Timing	Diners build a pre-order against their reservation; the kitchen times preparation so food is ready on arrival.	UC-03, UC-04 Traces: FR-002, FR-003	2.1 (5) · 2.2 (8) 2 stories, 13 pts — all Done
Epic 3 Payments & Promotions	Secure advance payment collection, promo-code redemption, and protection of diner payment/personal data.	UC-05, UC-07 Traces: FR-004, NFR-002	3.1 (5) · 3.2 (3) · 3.3 (5) 3 stories, 13 pts — all Done
Epic 4 Manager Floor Operations	Restaurant managers get live control of table status and layout, with real-time sync across floor terminals.	UC-09 Traces: FR-005, NFR-001	4.1 (5, Done) · 4.2 (3, In Progress) · 4.3 (8, In Progress) 3 stories, 16 pts — 5 done, 11 active

Backlog total: 12 stories · 55 story points across the 4 epics. 44 points are Done (Sprints 1–2); 11 points are active in Sprint 3.

2. Sprint Summary

Sprint	Planned window (IST)	Committed	Actual completion	Status
Sprint 1	Sep 1, 09:00 → Sep 2, 18:00	26 pts / 5 stories	Closed Sep 3, 14:31 (~20 h after planned end)	Done
Sprint 2	Sep 3, 09:00 → Sep 3, 22:00	18 pts / 5 stories	Closed Sep 4, 08:14 (~10 h after planned end)	Done
Sprint 3	Sep 4, 08:14 → Sep 4, 14:00	11 pts / 2 stories	Not yet complete	Active

3. Burndown Charts

Charts pulled from the Jira RTBPOA board report. Grey = guideline (ideal burn); red = actual remaining story points.

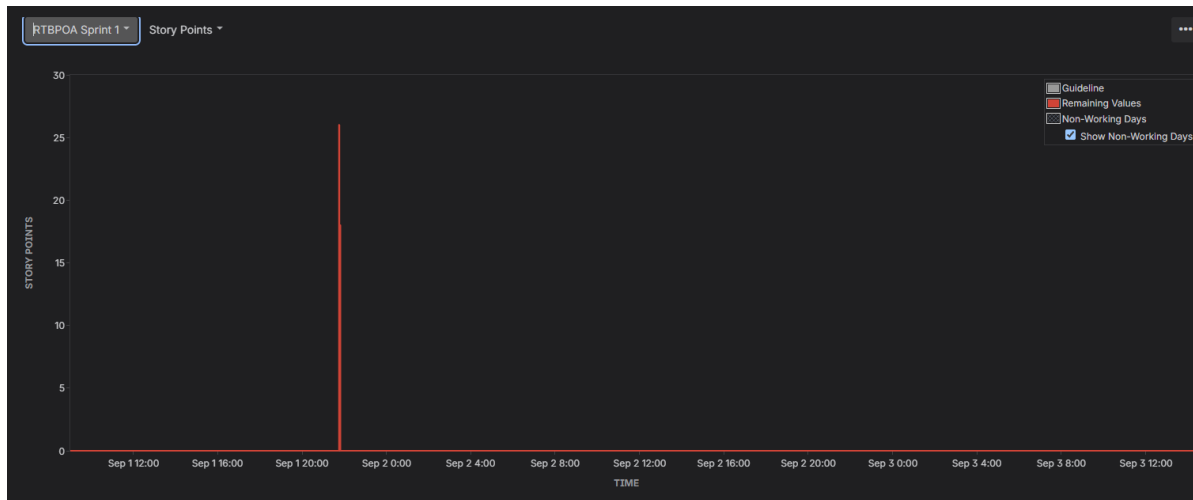


Figure 1 — Sprint 1 (26 pts). Full sprint window shown: the remaining-points line collapses to zero in a single vertical drop on the evening of Sep 1, well before the sprint was formally closed on Sep 3.

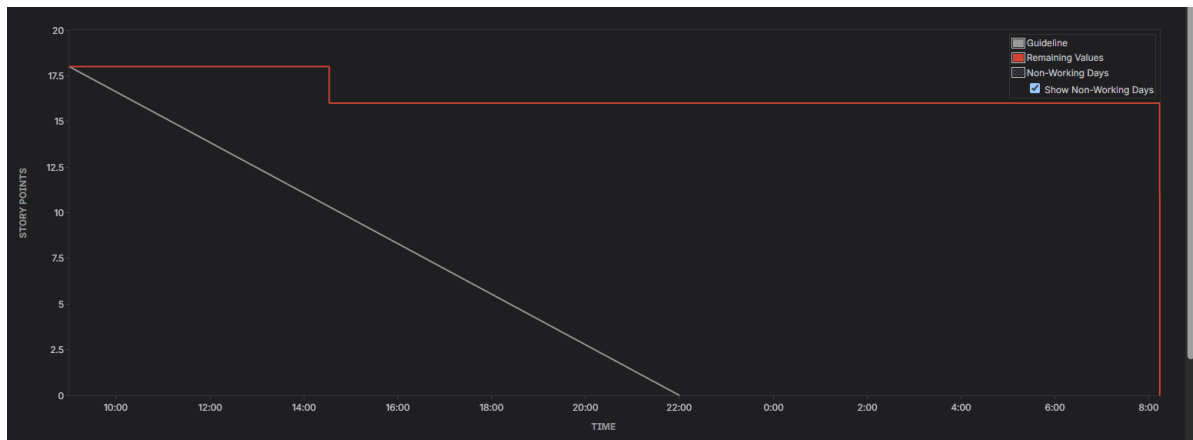


Figure 2 — Sprint 2 (18 pts). One story (~2–3 pts) burns down mid-afternoon on Sep 3, then the line goes flat — the remaining ~13 pts were not marked Done until after the visible window (closed Sep 4, 08:14).

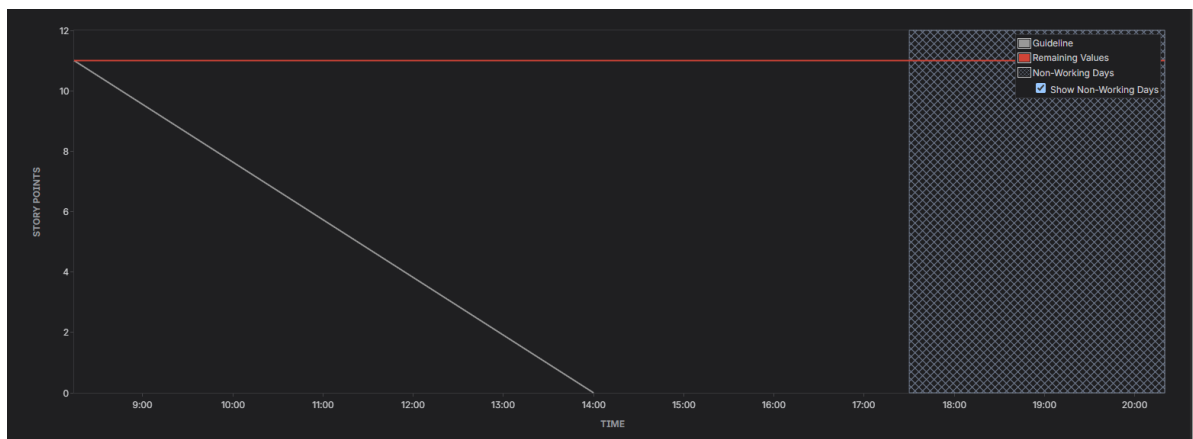


Figure 3 — Sprint 3 (11 pts, active). Remaining points are still flat at 11 against a guideline that should reach zero by 14:00 — no story has been marked Done yet in this sprint.

4. Reflection Questions

Draft answers below, grounded in the Jira data and burndown charts above — personalise before submitting.

1. Did your estimations reflect the actual effort?

Only partly. There is no logged time against any story (Original Estimate and Time Spent are unset throughout), so effort can't be checked hour-for-hour — but the sprint clocks are a useful proxy. In both closed sprints the work itself finished in a burst rather than a steady trickle: Sprint 1's 26 points collapsed to zero in one vertical drop on the evening of day one (Figure 1), and Sprint 2's remaining points sat flat until a late push closed it out. Yet neither sprint was formally closed within its planned window — Sprint 1 overran by roughly 20 hours and Sprint 2 by roughly 10. That gap points to the story-point estimates being reasonable for the coding effort itself, but the sprint time-boxes not accounting for review, verification, and admin overhead before a sprint could actually be closed.

2. Was your backlog well-prioritized?

Broadly yes. Sprint 1 front-loaded the diner-facing critical path — floor-plan browsing and table selection (Epic 1), the pre-order flow (Epic 2), and the first payment story (Epic 3) — which is the right order since a diner can't pre-order or pay before a table exists. Sprint 2 closed out the remaining reservation-management and payment/promotion stories and opened the first manager-side story (Epic 4). Manager floor operations were correctly left for last, since they support the diner flow rather than gate it. The one thing worth reconsidering: Story 2.2 (8 points, calculating the food-ready time) was the single largest story in Sprint 1 and arguably could have been split, since a large story sitting inside an otherwise fast-clearing sprint is the most likely source of the day-two-to-day-three closure lag.

3. How did your simulated sprint align with your plan?

Sprint 1 was planned as roughly a day and a half (Sep 1, 09:00 to Sep 2, 18:00) but wasn't closed until Sep 3, 14:31 — about 20 hours late, even though the burndown shows the actual work finishing within hours of the sprint starting. Sprint 2 was planned as a single day (09:00 to 22:00 on Sep 3) and ran about 10 hours over, closing at 08:14 the next morning. Sprint 3, the current active sprint, is planned as a short ~5h45m window (08:14 to 14:00 on Sep 4) and per Figure 3 has not burned down any points yet partway through that window. The pattern across all three: planned windows were tight relative to how long it actually took to get stories verified and marked Done.

4. What insights did the burndown chart give about your team's capacity?

The charts show a step/cliff pattern rather than the smooth diagonal of the ideal guideline in every sprint — work is completed in one or two concentrated bursts and marked Done all at once, not incrementally through the sprint. That's a reasonable simulation of how a small team actually works, but it also means the guideline is a poor predictor of daily progress for this team and burndown is only useful here as a completion marker, not an early-warning signal. Total throughput also held up across sprints (26, then 18, then 11 committed points), which suggests the team can reliably clear a sprint's backlog — the recurring issue is the sprint's time-box ending before the closing/verification step catches up with the finished work.